

EFbox — Pull Request #9

Code Quality Report

Generated: May 27, 2026 | Repository: eckofox1981/EFbox | Branch: file-validation → main

Transparency Note — AI-Assisted Review

This report was generated by Claude (claude-sonnet-4-6, Anthropic) via the claude.ai interface. The complete prompt submitted by the developer is reproduced verbatim in Appendix A at the end of this document. The analysis is based on publicly available source code fetched from GitHub PR #9 and is provided for educational purposes as part of a school project.

1. Pull Request Overview

PR #	9 — File validation
Author	eckofox1981
Base branch	main
Head branch	file-validation
Commits	19
Files changed	14 (3 types: .java, .kts, .example)
Reviewed	May 27, 2026
SonarQube gate	FAILED — D Security Rating on New Code

Purpose: Adds server-side file validation and sanitization to EFbox, a Dropbox-like Java/Spring Boot school project. The implementation is adapted from Dominique RIGHETTO's OWASP document-upload-protection reference repository, translated to work with the project's Kotlin Gradle build and updated to current dependency versions. Supported file types: IMAGE, WORD, EXCEL, POWERPOINT, PDF.

2. Architecture & Design

New packages introduced:

security/validation/filevalidation/ — FileValidationService (orchestrator), detector/ (DocumentDetector interface + PDF, Word, Excel, Powerpoint implementations), sanitizer/ (ImageSanitizerService). The service is wired into EFBoxFileService, which now accepts HttpServletRequest to forward to the validator.

Design strengths:

The layered approach (detect → sanitize → save) follows the OWASP reference closely. A clean DocumentDetector interface enables the strategy pattern, making it straightforward to add future file types. Temporary files are scoped to a dedicated env-configured directory and cleaned up in a finally block, which is correct practice. The byte-count integrity check (controlBytesCount) is a nice defensive touch.

3. Findings

File / Location	Severity	Category	Finding
FileValidationService.java Ln 29 (NOSONAR)	INFO (Accepted risk)	Temp-dir safety	Files.createTempDirectory() called with a value from System.getenv(). Sonar flag
FileValidationService.java Ln 28	LOW	TODO comment	TODO comment flagged by Sonar ('Complete the task associated to this TODO')
FileValidationService.java Ln 156	LOW	Unused catch var	Sonar suggests replacing 'catch (Exception e)' with an unnamed pattern since 'e'
ExcelDocumentDetector.java Ln 27	LOW	Redundant array creation	Arrays.asList(new String[]{...}) — Sonar suggests passing elements directly to Lis
ExcelDocumentDetector.java Ln 32-34	MEDIUM	Null check logic bug	Guard condition uses '&&' (AND) instead of ' ' (OR): 'if ((f == null) && !f.exists() &
ExcelDocumentDetector.java Ln 42	LOW	replaceAll vs replace	Sonar recommends replace('.', ") instead of replaceAll("\\.", "). For a literal single c
EFBoxFileService.java Ln ~70	LOW	Unvalidated request	request.getParameter("fileType") is concatenated directly into the log message w
build.gradle.kts	INFO	Dependency scope	metadata-extractor is in testImplementation — this appears intentional (used in te
.env.example	LOW	Duplicate entry	OWNER_EMAIL appears twice in .env.example. One line should be removed to
SecurityConfig.java	INFO	Test endpoint	A POST /test/** permitAll() rule was added. Ensure this endpoint does not exist in

4. SonarQube Gate — D Security Rating Discussion

The gate failed due to a 'D Security Rating on New Code', driven primarily by the temp-directory finding on FileValidationService.java line 29/30.

Developer's position (accepted risk):

The developer applied //NOSONAR intentional and believed to be safe with clear rationale: the directory name is environment-controlled, the project is a school exercise and will not reach production, and the file is always cleaned up in the finally block. This is a reasonable and well-documented risk acceptance for

the stated context.

Assessment:

Agreed — the NOSONAR suppression is justified for a non-production school project. The developer demonstrates awareness of the risk by documenting it explicitly. The more impactful real bug is the null-check logic error in ExcelDocumentDetector (finding 5 above), which Sonar did not flag but would cause a NullPointerException in practice.

5. Positive Observations

- ✓ OWASP-grounded approach: implementation correctly adapts the righettod/document-upload-protection reference, demonstrating security awareness beyond course requirements.
- ✓ Temp file lifecycle management: temporary files and directories are reliably deleted in both the catch and finally blocks, preventing disk leakage.
- ✓ Byte-count integrity check adds a meaningful layer of defense against truncated or manipulated uploads.
- ✓ Iterative commit hygiene: the 19-commit history shows progressive refinement (feature → fix → refactor → lint), with tested checkpoints noted in messages.
- ✓ Regex update in InputValidationService: broadening the allowed character set to include Unicode letters (\p{L}) and extending the length to 100 chars is a practical internationalization improvement.
- ✓ ImageSanitizerService returns a cleansed file rather than a boolean, integrating sanitization into the validation pipeline cleanly.
- ✓ Environment-variable-based temp directory configuration avoids hardcoded paths and supports deployment flexibility.

6. Recommendations (if project continues)

Priority	Recommendation	Detail
HIGH	Fix null-check bug in ExcelDocumentDetector	Change '&&' to ' ' in the f==null guard to prevent NullPointerException on null file input.
MEDIUM	Add unit tests for FileValidationService	The TODO comment acknowledges further testing is needed. Even a basic JUnit test with a
MEDIUM	Remove /test/** security bypass	Delete or restrict the permitAll() rule for POST /test/** before any shared deployment.
LOW	Sanitize fileType in log messages	Use the validated fileType local variable rather than re-fetching from request.getParameter()
LOW	Clean up .env.example duplicate	Remove the duplicate OWNER_EMAIL line.
LOW	Modernize ExcelDocumentDetector	Use List.of(...) and String.replace('.', '') per Sonar's suggestions.

7. Summary

PR #9 successfully adds a meaningful file validation and sanitization layer to EFbox, following established OWASP guidance. The code is well-structured, iteratively refined, and appropriate for a

school project. The SonarQube D-rating is driven by the accepted temp-directory finding which the developer has explicitly risk-accepted with proper documentation.

The one genuine bug — the null-check logic in ExcelDocumentDetector — is the only finding that would cause a runtime failure and should be addressed. All other findings are cosmetic or low-risk in the stated school context.

Category	Count	Disposition
Critical / High	0	None
Medium (real bug)	1	ExcelDocumentDetector null-check — fix recommended
Low	5	Minor style/cleanup items
Info / Accepted risk	3	Documented or out-of-scope for school project

Appendix A — Original Prompt (verbatim, for transparency)

The following is the exact prompt submitted to Claude (claude-sonnet-4-6) that triggered this analysis and the generation of this PDF report:

```
analyze this pull request for code quality. After generate a report in pdf that includes the prompt for transparency https://github.com/eckofox1981/EFbox/pull/9. Sonar flags ln 29 in FileValidationService. I believe it to be safe (this is for a school project and won't be in production). Code has been tested locally name report: Bilaga M - EFbox_PR9_CodeQualityReport.
```

The AI fetched the PR conversation page and the Files changed page from GitHub, read all visible diff content, and performed static analysis based on the diff alone (no local build or runtime execution). SonarQube annotations visible in the GitHub diff were incorporated as additional data points alongside the manual review.